

Ex. No.: 11c)

Date: 23/04/2025

Optimal

Aim:

To write a c program to implement Optimal page replacement algorithm.

ALGORITHM:

1. Start the process
2. Declare the size
3. Get the number of pages to be inserted
4. Get the value
5. Declare counter and stack
6. Select the least frequently used page by counter value
7. Stack them according the selection.
8. Display the values
9. Stop the process

PROGRAM:

```
#include <stdio.h>
```

```
int findOptimal (int pages[], int frames[], int currentIndex,  
                int total pages, int total frames){  
    int pos = -1, farthest = currentIndex;  
    for (int i = 0; i < total frames; i++) {  
        int j;  
        for (j = currentIndex + 1; j < total pages; j++) {  
            if (frames[i] == pages[j]) {  
                if (j > farthest) {  
                    farthest = j;  
                    pos = i;  
                }  
            }  
        }  
    }  
}
```



```

        break;
    }
    if (i == totalPages) {
        return i;
    }
    if (pos == -1) {
        pos = 0;
    }
    return pos;
}

```

```

int main() {
    int framesCount, pageCount;
    int pages[50], frames[10], faults = 0;
    printf("Enter number of frames: ");
    scanf("%d", &framesCount);
    printf("Enter number of pages: ");
    scanf("%d", &pageCount);
    printf("Enter the page reference string: ");
    for (int i = 0; i < pageCount; i++) {
        scanf("%d", &pages[i]);
    }
    for (int i = 0; i < framesCount; i++) {
        frames[i] = -1;
    }
    for (int i = 0; i < pageCount; i++) {
        int found = 0;
        for (int j = 0; j < framesCount; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
    }
}

```



```

if (!found) {
    int replaceIndex = -1;
    for (int j = 0; j < framesCount; j++) {
        if (frames[j] == -1) {
            replaceIndex = j;
            break;
        }
    }
    if (replaceIndex == -1) {
        replaceIndex = findOptimal(pages, frames, i, pagesCount,
            framesCount);
        frames[replaceIndex] = pages[i];
        faults++;
    }
    printf("Frame after page %d: ", pages[i]);
    for (int j = 0; j < framesCount; j++) {
        if (frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
    printf("\n");
    printf("Total Page faults: %d\n", faults);
    return 0;
}

```

Enter the number of frames: 3

Enter number of pages: 12

Enter the page reference string: 7 0 12 0 3 0 4 2 3
0 3

Output:

Frame after page 7: 7 - -
 Frame after page 0: 7 0 -
 Frame after page 1: 7 0 1
 Frame after page 2: 0 1 2
 Frame after page 0: 0 1 2
 Frame after page 3: 0 2 3
 Frame after page 0: 0 2 3
 Frame after page 4: 0 3 4
 Frame after page 2: 0 2 4
 Frame after page 3: 2 3 4
 Frame after page 0: 3 0 4
 Frame after page 3: 0 3 4
 Total Page Faults: 9

Result:

This Q. 11 the code to implement optimal page
 replacement algorithm has been executed
 successfully